Categorical Outcomes

Unit 11: Rates, Tables, and Predicting a Yes or No

Stat 220 | Statistics for Data Science

Where this fits

- Part I predicted *numbers*. An enormous share of real data is not a number: converted or not, churned or not, fraud or not, which region, which plan.
- This unit needed Part II first. Odds, base rates, and predictive values are conditional probability wearing different hats, and we only built that in Unit 7.
- Three arcs: describing rates honestly, comparing them in tables, and modeling a yes or no and turning it into a decision.

Principle: a rate is a fraction with a story on both floors

Every rate has a numerator and a denominator, and most arguments about rates are really arguments about the denominator.

Live experiment: the rock-paper-scissors tournament

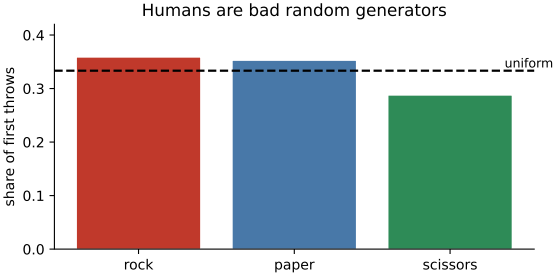
Live demo: can you be random?

Pair up. Play **five rounds** of rock-paper-scissors. One person in each pair records *only the first throw* of each round for both players.

- We will collect every first throw on the board as three counts.
- Then we test the hypothesis everyone assumes: that people throw each option one third of the time.

Materials: none. Time: 8 minutes including tallying the throws. Winners of the most rounds, keep your hand up. We will come back to whether you were lucky or good.

Stay here while they play: what the research says



Across large studies of first throws:

- **Rock** is thrown about 36% of the time, scissors only about 29%.
- So "always play paper" beats a typical human opponent slightly more often than it loses.

The edge is small, but it is free, and it exists because people cannot generate randomness.

Testing our own counts

Suppose the class produced n throws with counts $O_{\text{rock}}, O_{\text{paper}}, O_{\text{scissors}}$. Under "all equally likely," each expected count is $E = n/3$.

$$\chi^2 = \sum \frac{(O - E)^2}{E}, \qquad \text{compare to a chi-square with 2 degrees of freedom.}$$

- Bigger than about **6** means the pattern is hard to explain by chance.
- This is the same signal-to-noise logic as always: how far are the observed counts from what the null predicts, measured in units of how far they normally wander?

What this unit gives you

Things to know

- how to estimate a proportion and put an honest interval on it
- the three ways to express a comparison, and how differently they read
- contingency tables and chi-square, and what they do *not* say
- why denominators and group mix can reverse a conclusion
- logistic regression, log-odds, and odds ratios
- precision, recall, ROC/AUC, thresholds, and calibration

Mistakes to stop making

- quoting a relative change with no base rate
- confusing percentage points with percent
- reading a significant chi-square as a large or causal effect
- pooling across a variable that drives the outcome
- reporting accuracy on imbalanced data
- leaving the threshold at 0.5 because it is the default

One proportion, with honest uncertainty

$$\hat{p} = \frac{x}{n}, \qquad \mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \qquad \hat{p} \pm 1.96\,\mathrm{SE}.$$

- 12 conversions out of 300: $\hat{p} = 0.04$, SE $= 0.011$, interval roughly 1.8% to 6.2%.
- That is a *wide* interval, and reporting "our conversion rate is 4%" hides it completely.

Principle: a percentage is an estimate too

Every rate you have ever seen on a dashboard has a standard error. Most dashboards do not show it, and most arguments about week-over-week movement are arguments about noise.

When the count is small, patch the interval

- With few successes the standard formula misbehaves badly. Zero successes gives $\mathrm{SE} = 0$ and an interval of zero width, which is obviously wrong.
- Simple fix (the **plus-four** interval): add 2 successes and 2 failures, then use the usual formula. It works remarkably well.
- Zero out of 20 does not mean "never." A useful rough bound: with 0 events in n trials, the rate could plausibly be as high as $3/n$.

In practice: a question that catches people

"We saw no failures in 50 tests. What is our failure rate?" The good answer: "Not zero. The data is consistent with anything up to about 6%."

Comparing two proportions: three languages

A campaign moves conversion from 2.0% to 2.4%.

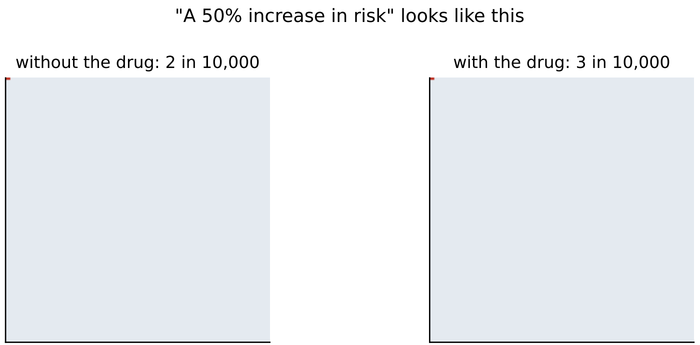
- **Absolute difference:** $+0.4$ *percentage points*.
- **Relative difference:** $+20\%$ (a fifth more).
- **Odds ratio:** about 1.21.

All three are true. They will produce three different meetings.

Trap: percent versus percentage points

Going from 2% to 2.4% is a rise of 0.4 percentage points *or* 20 percent. Writing "a 20% increase" when you meant 0.4 points, or the reverse, is the single most common numerical error in business writing.

Relative risk without a base rate is not usable



"This drug increases your risk by 50%" describes the move from 2 in 10,000 to 3 in 10,000. Both squares look empty because both risks are tiny. The headline is accurate and useless.

Your turn #1

Your turn: translate the headline

A news story reports: "Eating processed meat daily raises your risk of colorectal cancer by 18%." The lifetime risk without it is about 5%.

With a neighbor: what is the risk *with* daily processed meat, in absolute terms? How would you write the headline honestly, and would you change your diet?

Your turn #1: answer

- A relative increase of 18% on a 5% baseline gives about **5.9%**: roughly one extra case per 100 people.
- Honest version: "about 5 in 100 people get colorectal cancer, and among daily processed-meat eaters it is about 6 in 100."
- Whether to change your diet is now a real decision with real numbers, instead of a reaction to the word "raises."

Principle: always report both

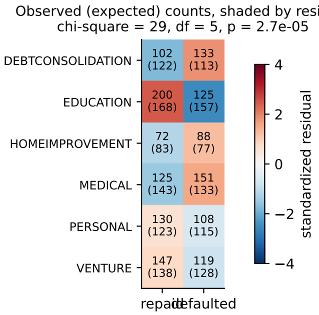
Give the relative change *and* the absolute change. One without the other is a choice about how alarmed you want your audience to be.

The contingency table

Cross-tabulate two categorical variables and ask whether they are related.

- **Null hypothesis:** the two variables are independent, so the row percentages should be the same in every row.
- **Expected count** for a cell: $\dfrac{(\text{row total})(\text{column total})}{\text{grand total}}$, which is exactly what independence implies.
- **Chi-square** adds up $(O - E)^2/E$ over all cells, with $(\text{rows} - 1)(\text{columns} - 1)$ degrees of freedom.

A real table: loan purpose and default



1,500 consumer loans.

- Education loans default far less than expected. Debt consolidation and medical loans default far more.
- The test says the association is real ($p \approx 3 \times 10^{-5}$).
- It does *not* say the purpose causes the default, or how large the effect is.

What a chi-square test does not tell you

- **Not the size** of the association. With 100,000 rows, a trivial dependence produces an enormous chi-square.
- **Not the direction or the pattern.** Follow up with the residuals to see which cells drive it, as in the figure.
- **Not causation.** It is an association in observational data, subject to every confounder you have not measured.

Trap: significance is not strength

Report the actual rates alongside the test. "Education loans default at 38% versus 57% for debt consolidation" is informative. "$p < 0.001$" by itself is not.

Assumptions and the small-cell problem

- The chi-square approximation needs reasonably large **expected** counts, roughly 5 or more in most cells.
- With sparse tables, use Fisher's exact test or combine categories that belong together.
- Every observation must be an independent unit. A table built from repeated measurements on the same customers overstates the evidence dramatically.

Trap: the cell-hunting expedition

A 6-by-4 table has 24 cells. Scanning them for the "significant" one is 24 tests, and something will always look striking. Decide what you are testing before you look.

Counts mislead. Rates need the right denominator

- "Region A had 340 complaints, Region B had 120." Region A also has four times the customers.
- The denominator should be the **exposure**: customers, shipments, patient-days, miles driven, hours of use.
- Choosing the denominator is a modeling decision. Accidents per driver, per mile, and per trip can rank the same drivers differently.

In practice: a good instinct to show

"Before I compare those numbers I want to know what each one is divided by, and whether the two denominators mean the same thing."

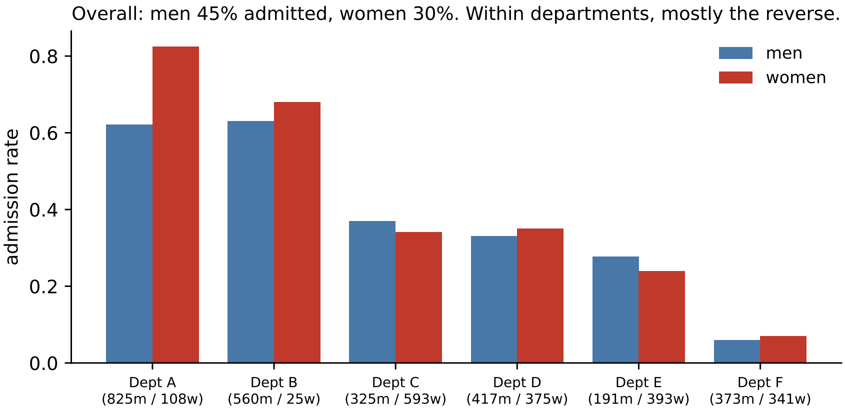
Standardization: comparing apples to apples

- Hospital A has a higher death rate than Hospital B. Hospital A is a trauma center that receives the sickest patients.
- The fix is **standardization**: compute the rate within each risk group, then combine using a common population mix.
- This is how age-adjusted mortality rates, risk-adjusted surgical outcomes, and seasonally-adjusted economic figures are built.

Principle: adjust before you rank

Any ranking of units that serve different populations is a ranking of the populations until you adjust for the mix.

Simpson's paradox, with the famous real case



Berkeley graduate admissions, 1973. Overall, 44% of men and 30% of women were admitted, and the university was sued. Department by department, women fared similarly or better. The difference: women applied far more often to the departments that admitted almost nobody.

Why the reversal happens, and what to do

- Aggregating mixes together groups with very different base rates and very different sizes. The pooled number is a weighted average with the wrong weights.
- The paradox appears whenever a **lurking variable** (here, department) affects both the group assignment and the outcome.
- The remedy is to *disaggregate*, or to model the outcome with the lurking variable included, exactly as you learned to do with a confounder in regression.

Trap: the aggregated conclusion

Any single number computed across heterogeneous groups can be reversed by regrouping. Before you present one, ask what happens within the obvious subgroups.

Your turn #2

Your turn: find the lurking variable

A hospital reports that patients treated by Surgeon X have a 12% complication rate, versus 6% for Surgeon Y. Administration proposes retraining Surgeon X.

With a neighbor: name two variables that could reverse this comparison, and state what you would compute instead.

Your turn #2: answer

- **Case severity** is the obvious one: the best surgeon often gets the hardest cases, which guarantees a worse raw rate.
- **Volume and case mix**: emergency versus scheduled, patient age, comorbidities.
- What to compute: complication rates *within* severity strata, or a model that adjusts for the risk factors, plus intervals so you can see whether 12% and 6% are even distinguishable given the number of operations.

Principle: the referral pattern is part of the data

Whenever units receive different inputs, the raw output comparison measures the inputs.

Why not just fit a line?

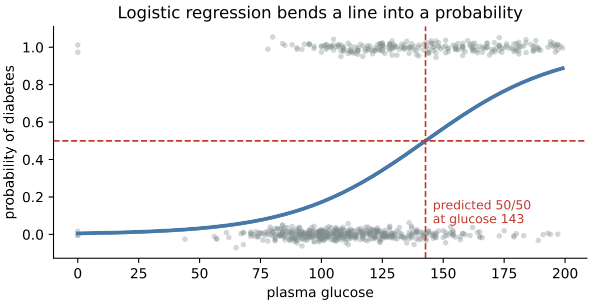
- If y is 0 or 1, a straight line predicts probabilities above 1 and below 0, which is nonsense.
- Logistic regression fits a line on the **log-odds** scale and then bends it into the 0-to-1 range:

$$\log \frac{p}{1-p} = b_0 + b_1 x_1 + \cdots + b_k x_k \qquad \Longleftrightarrow \qquad p = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + \cdots)}}$$

Principle: same skeleton as regression

Predictors combine linearly, coefficients come with standard errors, and the estimate-over-SE template still tells you what is real. Only the link to the outcome changed.

The curve, on real patient data



733 patients, predicting a diabetes diagnosis from plasma glucose.

- Each dot is a person at 0 or 1 (jittered so you can see them).
- The curve is the fitted probability. It is steep in the middle and flat at the ends, which is exactly how evidence should work.

Reading a logistic coefficient out loud

- A coefficient b_1 is a change in **log-odds** per unit of x. Nobody thinks in log-odds.
- Exponentiate it: e^{b_1} is an **odds ratio**. If $e^{b_1} = 1.35$, each extra unit multiplies the odds by 1.35, a 35% increase in the odds.
- Rough translation for small effects: divide the coefficient by 4 to get the maximum change in probability per unit.

Trap: odds are not probabilities

"The odds tripled" does not mean "three times as likely" in probability terms. Going from 1% to 3% and from 30% to 56% are both a tripling of odds, and they feel completely different to the business.

Live experiment: the inbox game

Live demo: you are the spam filter

Each team gets the same list of 20 emails, with the model's spam score for each (0 to 1) and a short description. Your job: pick **one threshold**. Everything at or above it goes to the spam folder.

- Missing a spam email (it lands in the inbox): costs you **1 point**.
- Blocking a real email (a job offer in the spam folder): costs you **10 points**.

Write your threshold on the board. Lowest total cost wins.

Materials: the email table on the next slide. Time: 10 minutes. Do not compute anything yet. Pick the number your gut says, then we will score it.

The inbox: 20 emails and the model's spam score

score	subject
0.97	Nigerian prince, urgent transfer
0.95	FREE CRYPTO GIVEAWAY!!!
0.93	Refinance now, rates expire
0.91	Recruiter: internship at Adobe
0.88	You have won a gift card
0.84	Weight loss miracle
0.79	Career fair: employers attending
0.74	Invoice attached, unknown vendor
0.68	Professor: exam rescheduled
0.61	Limited time software offer

score	subject
0.55	Bank: unusual sign-in detected
0.49	Newsletter you never wanted
0.44	Landlord: lease renewal
0.38	Group project meeting Thursday
0.31	Conference invite, pay to publish
0.26	Airline: flight delayed
0.19	Mom: call me
0.13	GitHub: pull request review
0.08	Bill due Friday
0.03	Slack digest

One threshold per team. Miss a spam: 1 point. Block a real email: 10 points.

Stay here while they choose: what will happen

- Teams that picked **0.5** because it is the default will do poorly.
- Because blocking real mail is ten times worse, the winning threshold will be high, somewhere around **0.9**: only block when you are very sure.
- Now flip the costs. If we were filtering *phishing attacks in a hospital*, missing one is catastrophic and the threshold should be low.

Scoring the inbox game

threshold	real emails blocked	spam missed	**total cost**
0.30	6	0	60
0.50 (the default)	4	2	**42**
0.70	2	3	23
0.80	1	4	14
0.92	**0**	**6**	**6**
1.00 (block nothing)	0	9	9

Scoring the inbox game

threshold	real emails blocked	spam missed	**total cost**
0.30	6	0	60
0.50 (the default)	4	2	**42**
0.70	2	3	23
0.80	1	4	14
0.92	**0**	**6**	**6**
1.00 (block nothing)	0	9	9

Principle: the threshold is a business decision, not a modeling one

The best threshold here is **0.92**, seven times cheaper than the 0.5 default. The model never changed. Only the number you compared it to did, and that number came from the *costs*, not from the data.

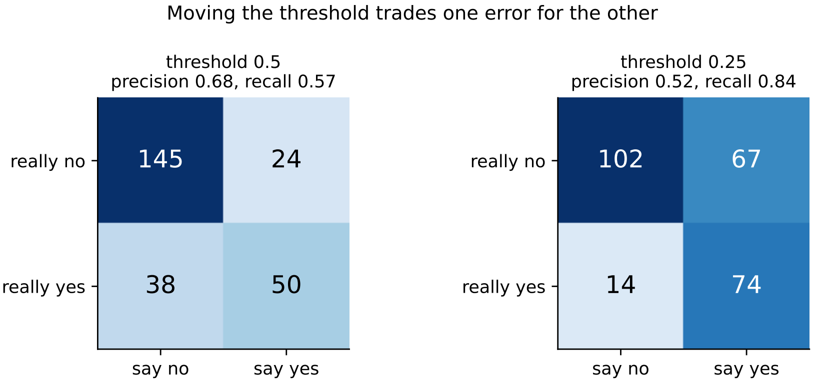
The confusion matrix, and the two ways to be wrong

- **True positive:** you said yes and it was yes.
- **False positive:** you said yes and it was no. A false alarm.
- **False negative:** you said no and it was yes. A miss.
- **True negative:** you said no and it was no.

$$\text{precision} = \frac{TP}{TP + FP} \quad \text{(of those I flagged, how many were real?)}$$

$$\text{recall} = \frac{TP}{TP + FN} \quad \text{(of the real ones, how many did I catch?)}$$

Two thresholds on the same model



Same model, same test set, one number changed. Lowering the threshold from 0.5 to 0.25 catches many more real cases (recall rises) at the price of more false alarms (precision falls). No *threshold* improves both at once. Only a better model does, which is why the threshold and the model are separate decisions.

Which one do you care about?

- **Recall matters most** when a miss is catastrophic: cancer screening, fraud on a large account, safety alerts.
- **Precision matters most** when acting is expensive or annoying: sending a technician, blocking a customer's card, flooding a review queue.
- Almost every real system needs a stated tradeoff, not a single metric.

You may be asked: answer this way

"It depends on the cost of each error. If missing a fraudulent charge costs \$400 and reviewing a false alarm costs \$3, I want recall, and I would set the threshold from that ratio."

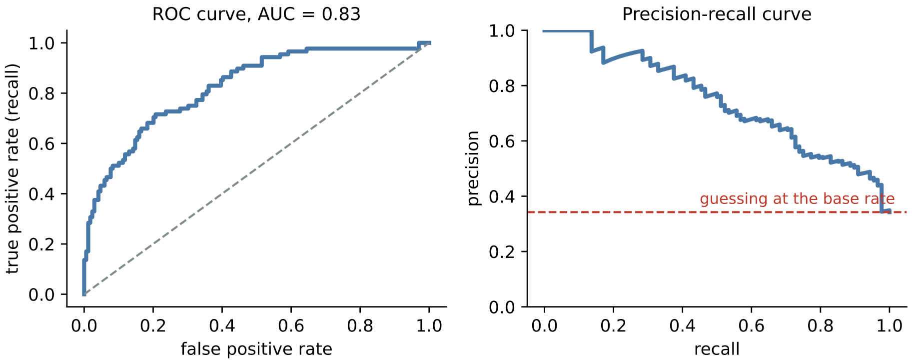
Why accuracy lies

- In our test set, 34% of patients have diabetes. A model that predicts "no" for everyone scores **66% accuracy** and is worthless.
- Make the disease rarer, say 1 in 1000, and "always no" scores **99.9%**.
- Accuracy rewards predicting the majority class, which is exactly what you do not need a model for.

Trap: the interview trap you will definitely be asked

"Our model is 99% accurate." The correct first question is always: *what is the base rate, and what does the trivial model score?*

ROC and precision-recall curves



The ROC curve sweeps every threshold at once, and AUC is the area under it (0.83 here). AUC is the chance the model ranks a random positive above a random negative. On imbalanced problems the precision-recall curve is far more informative, because it shows what fraction of your flagged queue is real.

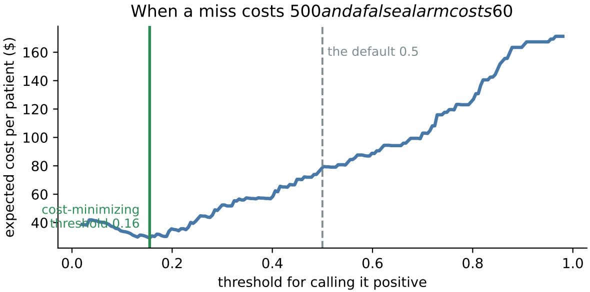
What AUC does and does not tell you

- **Does:** summarize ranking quality across all thresholds, independent of the base rate. Good for comparing two models.
- **Does not:** tell you whether the model is useful, what threshold to use, whether the probabilities are honest, or whether the flagged queue is workable.
- A model with AUC 0.95 can still produce a queue that is 90% false positives if the event is rare enough.

Principle: report the operating point, not just the curve

"AUC 0.87" is a modeling statistic. "At our threshold we catch 62% of fraud and 1 in 5 flags is real" is an answer someone can act on.

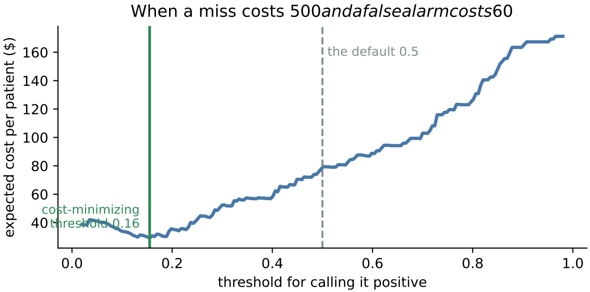
The threshold follows from the costs



Missing a case costs $500 and an unnecessary follow-up costs $60.

- The cost-minimizing threshold is about **0.16**, not 0.5.
- Using the default would nearly double the expected cost per patient.

The threshold follows from the costs



Missing a case costs \$500 and an unnecessary follow-up costs \$60.

- The cost-minimizing threshold is about **0.16**, not 0.5.
- Using the default would nearly double the expected cost per patient.

Rule of thumb: the optimal threshold is near $\text{cost}_{FP}/(\text{cost}_{FP} + \text{cost}_{FN})$.

Where 0.5 comes from, and why it is usually wrong

- 0.5 is optimal only when the two errors cost exactly the same. That is almost never true.
- It is a software default, not a decision. Every serious deployment tunes it.
- Tune it on **held-out** data, using real costs, and re-tune when the costs or the base rate change.

You may be asked: a question that separates candidates

"Where would you set the threshold?" A weak answer names a number. A strong answer asks for the two costs and the base rate first, then names a number.

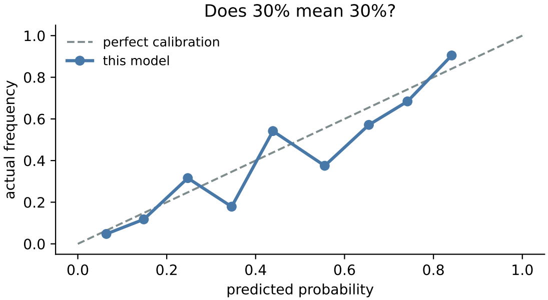
Class imbalance, handled honestly

- Imbalance is a fact about the world, not a modeling bug. Fraud really is rare.
- Reasonable responses: use precision-recall rather than accuracy, tune the threshold, weight the classes in the loss, or gather more positives.
- Resampling (oversampling positives, undersampling negatives) can help the fit, but it **distorts the predicted probabilities**. If you resample, you must recalibrate before anyone reads a probability.

Trap: the balanced-data trap

A model trained on a 50/50 rebalanced sample will predict roughly 50% probabilities in a world where the true rate is 1%. It ranks fine and its probabilities are fiction.

Calibration: does 30% mean 30%?



Group the predictions into bins and check: among cases predicted at 30%, did about 30% actually happen?

- On the diagonal: calibrated, and the number can be used in an expected-value calculation.
- Off the diagonal: the ranking may still be fine, but the probabilities cannot be multiplied by dollars.

Your turn #3

Your turn: set the threshold, defend it

You build a model to flag transactions for manual review. Reviewing one costs \$4 of analyst time. A missed fraud costs an average of \$220. Fraud is 0.3% of transactions.

With a neighbor: roughly where should the threshold go, and what is the one thing you must check about the model before that number means anything?

Your turn #3: answer

- The cost ratio gives a threshold around $4/(4+220) \approx \mathbf{0.018}$. You review anything above roughly a 2% chance of fraud.
- Most of that queue will be honest transactions, and that is correct: it is worth four dollars to avoid a one-in-fifty chance of losing $220.
- The thing to check first: **calibration**. A threshold in probability units is meaningless if the model's "2%" is not really 2%.

Principle: expected value, one transaction at a time

Flag when $P(\text{fraud}) \times$ cost of a miss $>$ cost of a review. Every threshold decision is that comparison.

Scenario: the model that was too good

Setup. A hospital model predicts which patients will be readmitted, with AUC 0.99 in testing. In production it is useless.

- Suspect **leakage** first. Something in the training data encoded the answer: a discharge code entered after readmission, or a billing field only filled in later.
- Check the timing of every feature: was this value knowable *at the moment of prediction*?
- The AUC itself was the warning. Real clinical prediction lands around 0.7 to 0.85.

Trap: amazing performance is a symptom

Your first reaction to a near-perfect classifier should be to hunt for leakage, not to celebrate.

LLM check: mistake or not? (1 of 2)

LLM check: we asked an LLM to interpret an A/B test

We told it: "Version A converted 20 of 500 visitors, version B converted 28 of 500. Is B better?"

It answered: "Yes. B's conversion rate is 5.6% versus A's 4.0%, a 40% improvement. That is a substantial lift and you should roll out version B."

Mistake, or no mistake?

LLM check: mistake or not? (1 of 2)

LLM check: we asked an LLM to interpret an A/B test

We told it: "Version A converted 20 of 500 visitors, version B converted 28 of 500. Is B better?"

It answered: "Yes. B's conversion rate is 5.6% versus A's 4.0%, a 40% improvement. That is a substantial lift and you should roll out version B."

Mistake, or no mistake?

Trap: verdict: mistake

The difference is 1.6 percentage points with standard errors near 1 point each. A two-proportion test gives $p \approx 0.25$: entirely consistent with chance. The "40% improvement" framing makes noise sound like a result.

LLM check: mistake or not? (2 of 2)

LLM check: same territory, a different answer

We told it: "We oversampled the minority class to 50/50 and our model now predicts 45% churn probability for most customers. Is that a problem?"

It answered: "Yes. Rebalancing changes the base rate the model learned, so the predicted probabilities no longer reflect reality even if the ranking improved. If you need probabilities for expected-value decisions, recalibrate on a sample with the true class rate."

Mistake, or no mistake?

LLM check: mistake or not? (2 of 2)

LLM check: same territory, a different answer

We told it: "We oversampled the minority class to 50/50 and our model now predicts 45% churn probability for most customers. Is that a problem?"

It answered: "Yes. Rebalancing changes the base rate the model learned, so the predicted probabilities no longer reflect reality even if the ranking improved. If you need probabilities for expected-value decisions, recalibrate on a sample with the true class rate."

Mistake, or no mistake?

Principle: verdict: no mistake

It separated ranking quality from probability honesty, which is the distinction this module is built around.

The arc of a classification project

1. **Name the decision**, not the label. What action follows a "yes"?
2. **Get the base rate** before modeling anything.
3. **Write down the two costs**, even roughly. This determines everything downstream.
4. **Fit and evaluate out of sample**, with precision, recall, and a PR curve.
5. **Choose the threshold from the costs**, on held-out data.
6. **Check calibration** if anyone will multiply a probability by a dollar amount.
7. **Report the operating point** in business language, with the queue size it implies.

From a rate to a decision

1. **Write the rate as a fraction** and say out loud what the denominator is.
2. **Attach an interval** to every rate you plan to compare: `proportion_confint(x, n, method="wilson")`.
3. **Compare two rates** with `proportions_ztest`, and report the absolute difference, the relative change, and the odds ratio.
4. **For a table**, compute expected counts and run `stats.chi2_contingency`, then read the residuals to see which cells drive it.
5. **Disaggregate** by the obvious lurking variable before publishing a pooled number.
6. **For a model**, fit `LogisticRegression` and exponentiate the coefficients to get odds ratios.
7. **Choose the threshold from the two error costs**, then report the operating point in plain language.

The traps, all in one place

Trap: the ones that bite most often

- Reporting a relative change with no base rate, or points confused with percent.
- Comparing counts across groups of different size.
- Treating a significant chi-square as a large or causal effect.
- Pooling across a variable that drives both the group and the outcome.
- Reporting accuracy when the classes are imbalanced.
- Leaving the threshold at 0.5, or treating the two errors as equally costly.
- Multiplying an uncalibrated "probability" by dollars.
- Believing a near-perfect classifier before checking for leakage.

Ten questions to be able to answer

1. "This doubles your risk." What do you ask next?
2. Conversion moved from 2.0% to 2.4%. How would you decide whether it is real?
3. We saw zero failures in 50 tests. What is the failure rate?
4. What does a chi-square test tell you, and what does it not?
5. How can a treatment look better in every subgroup and worse overall?
6. A model is 99% accurate on a 1-in-1000 event. What do you ask next?
7. Define precision and recall, and give a case where each is the priority.
8. Where do you set the threshold, and what do you need to know first?
9. How do you interpret a logistic coefficient of 0.8?
10. What is calibration, how would you check it, and when does it matter?

Mastery checklist

You have this unit when you can, from memory:

- estimate a proportion with an honest interval, even with very few events
- express a comparison as an absolute difference, a relative change, and an odds ratio
- build a contingency table, compute expected counts, and read the residuals
- choose and defend a denominator, and standardize when the group mix differs
- recognize and resolve Simpson's paradox in a real table
- write the logistic model and interpret a coefficient as an odds ratio
- compute precision and recall, and explain why accuracy fails under imbalance
- derive a threshold from the two error costs and state the operating point in plain business language

What you should be able to do with software

Nobody is asking you to memorize syntax. You are expected to know *which* procedure the question calls for, what to hand it, and what the output means. With an AI assistant open, you should be able to produce any of these and defend the result.

You should be able to	What you would reach for
Interval for a single proportion	`proportion_confint(x, n, method="wilson")`
Compare two proportions	`proportions_ztest([x1, x2], [n1, n2])`
Contingency table and chi-square	`pd.crosstab`, `stats.chi2_contingency`
Logistic regression and odds ratios	`LogisticRegression`, then `np.exp(coef)`
Confusion matrix, precision, recall	`confusion_matrix`, `classification_report`
ROC, AUC, and precision-recall	`roc_curve`, `auc`, `precision_recall_curve`
Sweep the threshold on cost	loop over thresholds and total the two error costs

If you cannot say what the output means, the fact that you produced it is worth nothing.

Where we go next

- Every model in Units 5 through 8 answers "what tends to go with what." None of them answers "what happens if we change something."
- **Unit 5** takes that second question seriously, and shows exactly which assumptions you have to buy in order to earn a causal claim.

Principle: the habit to keep

A classifier does not make decisions. It supplies probabilities, and you supply the costs. Keeping those two jobs separate is most of the skill.